

Nama: Muhammad Akmal Dwiansyah Putra

Kelas / Absen: TI_1G / 20

NIM: 254107020110

Praktikum Jobsheet 10

Praktikum 1:

Queue.java:

```
package Jobsheet10;
```

```
public class Queue {
```

```
    int[] data;
```

```
    int front;
```

```
    int rear;
```

```
    int size;
```

```
    int max;
```

```
    Queue(int n){
```

```
        max = n;
```

```
        data = new int[max];
```

```
        size = 0;
```

```
        front = rear = -1;
```

```
    }
```

```
    boolean IsFull(){
```

```
        if (size == max) {
```

```
            return true;
```

```
        }else{
```

```
            return false;
```

```
        }
```

```
    }
```

```
boolean IsEmpty(){  
    if (size == 0) {  
        return true;  
    }else{  
        return false;  
    }  
}
```

```
void Enqueue(int dt){  
    if (IsFull()) {  
        System.out.println("Queue sudah penuh");  
    }else{  
        if (IsEmpty()) {  
            front = rear = 0;  
        }else{  
            if (rear == max - 1) {  
                rear = 0;  
            }else{  
                rear++;  
            }  
        }  
    }  
    data[rear] = dt;  
    size++;  
}
```

```
int Dequeue(){  
    int dt = 0;  
    if (IsEmpty()) {
```

```

        System.out.println("Queue masih kosong");
    }else{
        dt = data[front];
        size--;
        if (IsEmpty()) {
            front = rear = -1;
        }else{
            if (front == max - 1) {
                front = 0;
            }else{
                front++;
            }
        }
    }
    return dt;
}

void peek(){
    if (!IsEmpty()) {
        System.out.println("Elemen terdepan: " + data[front]);
    }else{
        System.out.println("Queue masih kosong");
    }
}

void print(){
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    }else{
        int i = front;

```

```

        while (i != rear) {
            System.out.print(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen: " + size);
    }
}

```

```

void clear(){
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    }else{
        System.out.println("Queue masih kosong");
    }
}
}

```

QueueMain:

```

package Jobsheet10;

import java.util.Scanner;

public class QueueMain {
    public static void menu(){
        System.out.println("Masukkan operasi yang diinginkan:");
        System.out.println("1. Enqueue");
        System.out.println("2. Dequeue");
    }
}

```

```

        System.out.println("3. Print");
        System.out.println("4. Peek ");
        System.out.println("5. Clear");
        System.out.println("-----");
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan kapasitas queue: ");
        int n = sc.nextInt();
        int pilih;

        Queue Q = new Queue(n);

        do {
            menu();
            pilih = sc.nextInt();
            switch (pilih) {
                case 1:
                    System.out.print("Masukkan data baru: ");
                    int dataMasuk = sc.nextInt();
                    Q.Enqueue(dataMasuk);
                    break;
                case 2:
                    int dataKeluar = Q.Dequeue();
                    if (dataKeluar != 0) {
                        System.out.println("Data yang dikeluarkan: " + dataKeluar);
                        break;
                    }
                case 3:

```

```

        Q.print();
        break;
    case 4:
        Q.peek();
        break;
    case 5:
        Q.clear();
        break;
    default:
        break;
    }

} while (pilih == 1 || pilih == 2 || pilih == 3 || pilih == 4 || pilih == 5);

sc.close();
}
}

```

Hasil:

```

Masukkan kapasitas queue: 4
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
1
Masukkan data baru: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
1
Masukkan data baru: 31
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
4
Elemen terdepan: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----

```

2.1.3. Pertanyaan

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?

Jawaban: Rear dan front bernilai -1 ini digunakan untuk mendeteksi apakah program memiliki nilai atau tidak, jika belum di isi nilai maka `IsEmpty == true`. Sedangkan size bernilai 0 karena digunakan untuk mendeteksi apakah nilai array sudah penuh atau belum dengan menggunakan `size = max - 1`.

2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {  
    rear = 0;
```

Jawaban: Potongan program tersebut digunakan jika rear dari array sudah mencapai batas dari max isi array. Dengan potongan kode tersebut rear akan dipindahkan ke depan atau dimulai dari 0.

3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {  
    front = 0;
```

Jawaban: Sama seperti rear pada Enqueue, program akan memindahkan front kedepan jika sudah mencapai batas dari isi array.

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (`int i=0`), melainkan `int i=front`?

Jawaban: Menggunakan `i=front` dari pada `i=0` karena tidak selalu isi dari array yang telah di input selalu terletak pada 0, jika melakukan dequeue maka bari terdepan menjadi 1. Ini mengapa menggunakan front karena melacak letak terdepan dari array bukan terdahulu.

5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

Jawaban: Potongan tersebut digunakan untuk mengembalikan nilai i menjadi 0. Ini hanya terjadi jika nilai `i + 1 == max`. itu kenapa menggunakan modulus dari max.

6. Tunjukkan potongan kode program yang merupakan queue overflow!

Jawaban:

```
boolean IsFull(){  
    if (size == max) {  
        return true;  
    }else{  
        return false;  
    }  
}
```

```
}
```

7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

Jawaban: while (!Q.IsEmpty() || Q.IsFull());

Praktikum 2:

Mahasiswa.java:

```
package Jobsheet10;
```

```
public class Mahasiswa {
```

```
    String nim;
```

```
    String nama;
```

```
    String prodi;
```

```
    String kelas;
```

```
    Mahasiswa(String nim, String nama, String prodi, String kelas){
```

```
        this.nim = nim;
```

```
        this.nama = nama;
```

```
        this.prodi = prodi;
```

```
        this.kelas = kelas;
```

```
    }
```

```
    void TampilkanData(){
```

```
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
```

```
    }
```

```
}
```

AntrianLayanan.java:

```
package Jobsheet10;
```



```
public class AntrianLayanan {  
    Mahasiswa[] data;  
    int front;  
    int rear;  
    int size;  
    int max;  
  
    boolean IsFull(){  
        if (size == max) {  
            return true;  
        }else{  
            return false;  
        }  
    }  
  
    boolean IsEmpty(){  
        if (size == 0) {  
            return true;  
        }else{  
            return false;  
        }  
    }  
  
    AntrianLayanan(int max){  
        this.max = max;  
        data = new Mahasiswa[max];  
        front = 0;  
        rear = -1;  
        size = 0;  
    }  
}
```

```

void TambahAntrian(Mahasiswa siswa){
    if (IsFull()) {
        System.out.println("Antrian penuh, tidak dapat menambahkan Mahasiswa");
        return;
    }
    rear = (rear + 1) % max;
    data[rear] = siswa;
    size++;
    System.out.println(siswa.nama + " berhasil ditambahkan");
}

```

```

Mahasiswa LayananMahasiswa(){
    if (IsEmpty()) {
        System.out.println("Antrian kosong");
        return null;
    }
    Mahasiswa siswa = data[front];
    front = (front + 1) % max;
    size--;
    return siswa;
}

```

```

void LihatTerdepan(){
    if (IsEmpty()) {
        System.out.println("Antrian Kosong");
    }else{
        System.out.print("Mahasiswa Terdepan: ");
        System.out.println("NIM - Nama - Prodi - Kelas");
        data[front].TampilkanData();
    }
}

```

```
}  
}
```

```
void TampilkanSemua(){  
    if (IsEmpty()) {  
        System.out.println("Antrian Kosong");  
        return;  
    }  
    System.out.println("Daftar Mahasiswa dalam Antrian");  
    System.out.println("NIM - Nama - Prodi - Kelas");  
    for (int i = 0; i < size; i++) {  
        int index = (front + i) % max;  
        System.out.print((i+1) + ". ");  
        data[index].TampilkanData();  
    }  
}  
  
int GetJumlahAntrian(){  
    return size;  
}  
}
```

LayananAkademikSIKAD.java:

```
package Jobsheet10;
```

```
import java.util.Scanner;
```

```
public class LayananAkademikSIKAD {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
AntrianLayanan antrian = new AntrianLayanan(5);
```

```
int pilihan;
```

```
do {
```

```
    System.out.println("\n=== Menu Antrian Layanan Akademik ===");
```

```
    System.out.println("1. Tambah Mahasiswa ke antrian");
```

```
    System.out.println("2. Layani Mahasiswa");
```

```
    System.out.println("3. Lihat Mahasiswa terdepan");
```

```
    System.out.println("4. Lihat semua antrian");
```

```
    System.out.println("5. Jumlah Mahasiswa dalam antrian");
```

```
    System.out.println("0. Keluar");
```

```
    System.out.print("Pilih menu: ");
```

```
    pilihan = sc.nextInt();
```

```
    sc.nextLine();
```

```
switch (pilihan) {
```

```
    case 1:
```

```
        System.out.print("NIM : ");
```

```
        String nim = sc.nextLine();
```

```
        System.out.print("Nama : ");
```

```
        String nama = sc.nextLine();
```

```
        System.out.print("Prodi: ");
```

```
        String prodi = sc.nextLine();
```

```
        System.out.print("Kelas: ");
```

```
        String kelas = sc.nextLine();
```

```
        Mahasiswa siswa = new Mahasiswa(nim, nama, prodi, kelas);
```

```
        antrian.TambahAntrian(siswa);
```

```
        break;
```

```
    case 2:
```

```

        Mahasiswa dilayani = antrian.LayananMahasiswa();
        if (dilayani != null) {
            System.out.print("Melayani Mahasiswa: ");
            dilayani.TampilkanData();
        }
        break;
    case 3:
        antrian.LihatTerdepan();
        break;
    case 4:
        antrian.TampilkanSemua();
        break;
    case 5:
        System.out.println("Jumlah dalam antrian: " + antrian.GetJumlahAntrian());
        break;
    case 0:
        System.out.println("Terima Kasih");
        break;
    default:
        System.out.println("Pilihan Tidak Valid");
    }
} while (pilihan != 0);
sc.close();
}
}

```

HASIL:

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa terdepan
4. Lihat semua antrian
5. Jumlah Mahasiswa dalam antrian
0. Keluar
Pilih menu: 1
NIM : 123
Nama : Aldi
Prodi: TI
Kelas: 1A
Aldi berhasil ditambahkan
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa terdepan
4. Lihat semua antrian
5. Jumlah Mahasiswa dalam antrian
0. Keluar
Pilih menu: 1
NIM : 124
Nama : Bobi
Prodi: TI
Kelas: 1G
Bobi berhasil ditambahkan
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa terdepan
4. Lihat semua antrian
5. Jumlah Mahasiswa dalam antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian
NIM - Nama - Prodi - Kelas
1. 123 - Aldi - TI - 1A
2. 124 - Bobi - TI - 1G

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa terdepan
4. Lihat semua antrian
5. Jumlah Mahasiswa dalam antrian
0. Keluar
Pilih menu: 2
Melayani Mahasiswa: 123 - Aldi - TI - 1A
```

```

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa terdepan
4. Lihat semua antrian
5. Jumlah Mahasiswa dalam antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian
NIM - Nama - Prodi - Kelas
1. 124 - Bobi - TI - 1G

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa terdepan
4. Lihat semua antrian
5. Jumlah Mahasiswa dalam antrian
0. Keluar
Pilih menu: 5
Jumlah dalam antrian: 1

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa terdepan
4. Lihat semua antrian
5. Jumlah Mahasiswa dalam antrian
0. Keluar
Pilih menu: 0
Terima Kasih

```

2.2.3 Pertanyaan

Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIKAD sehingga method LihatAkhir dapat dipanggil!

Jawaban:

LayananAkademikSIKAD.java:

case 6:

antrian.LihatTerbelakang();

break;

AntrianLayanan.java:

```

void LihatTerbelakang(){
    if (IsEmpty()) {

```

```
        System.out.println("Antrian kosong");
    }else{
        System.out.print("Mahasiswa Terbelakang: ");
        System.out.println("NIM - Nama - Prodi - Kelas");
        data[rear].TampilkanData();
    }
}
```